Graph-Based Contrastive Learning and Clustering for Open-World Encrypted Traffic Classification

Jigang Wen, Xiyu Zhang, Kun Xie, *Member, IEEE*, Yuxiang Zeng, Kun Huang, Yudian Ouyang, and Wei Liang

***Abstract*—Encrypted traffic classification is essential for modern network management and security, yet remains highly challenging due to payload encryption, data imbalance, and the emergence of unknown traffic classes. Existing methods often assume closed-world settings, rely on handcrafted features, or lack robustness in dynamic environments. To address these limitations, we propose a unified Graph-based Contrastive Learning and Clustering (GCLC) framework for encrypted traffic classification in open-world scenarios. Encrypted flows are modeled as traffic interaction graphs using lightweight packet-level features. We train a Graph Neural Network (GNN) encoder with a hybrid loss function, integrating supervised contrastive learning and center-aware loss to promote the aggregation of samples into class clusters with clear boundaries between clusters. To enhance robustness under limited and imbalanced data, we design a traffic-aware graph augmentation strategy that simulates the variability of real-world network conditions. To discover new classes, we introduce a Mahalanobis distance-based new class detector and an adaptive model update mechanism that can detect and integrate unseen traffic classes. GCLC is evaluated on 3 public datasets and 1 self-collected dataset. Experimental results show that it outperforms existing methods on 4 evaluation metrics, with an open-set new class recognition accuracy of 95.02%. Even under scenarios of severe class imbalance and unknown classes, it still maintains high performance, exhibits strong robustness and adaptability, confirming the effectiveness of the framework in real-world encrypted dynamic traffic environments.**

Received 18 November 2025; revised 9 May 2026; accepted 28 May 2026. Date of publication 18 June 2026; date of current version 14 July 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62321166652, Grant U25B2038, Grant U2468205, Grant 62472167, and Grant 62402172; in part by Hunan Provincial Natural Science Foundation of China under Grant 2024JJ3014 and Grant 2024JJ5165; and in part by the Project of Yuelushan Center for Industrial Innovation under Grant 2025YCI10125. The associate editor coordinating the review of this article and approving it for publication was Prof. Guowen Xu. *(Corresponding author: Xiyu Zhang.)*

Jigang Wen, Xiyu Zhang, and Wei Liang are with the School of Computer Science and Engineering, Hunan University of Science and Technology, Xiangtan 411201, China (e-mail: wenjigang@hnust.edu.cn; zhangxiyu@mail.hnust.edu.cn; wliang@hnust.edu.cn).

Kun Xie is with the College of Computer Science and Electronic Engineering, Hunan University, Changsha 410082, China, also with the Key Laboratory of Supercomputing and Artificial Intelligence Fusion Computing, Ministry of Education, Changsha 410082, China, and also with Yuelushan Center for Industrial Innovation, Changsha 410082, China (e-mail: xiekun@hnu.edu.cn).

Yuxiang Zeng is with the College of Computer Science and Electronics Engineering, Hunan University, Changsha 410082, China (e-mail: zengyuxiang@hnu.edu.cn).

Kun Huang is with Pengcheng Laboratory, Shenzhen 518055, China (e-mail: huangk03@pcl.ac.cn).

Yudian Ouyang is with Hunan Normal University, Changsha 410081, China (e-mail: yudian@hnu.edu.cn).

Digital Object Identifier 10.1109/TIFS.2026.3705313

***Index Terms*—Encrypted traffic classification, graph augmentation, contrastive learning, graph neural networks (GNNs), graph-level clustering.**

I. Introduction

ENCRYPTED traffic classification has become a cornerstone task in modern networking, enabling a wide range of applications such as intrusion detection, quality-of-service (QoS) provisioning, policy enforcement, and application-aware resource management. As more network services adopt encryption protocols like TLS, QUIC, and VPN to protect user privacy and circumvent network surveillance [1], [2], traditional classification methods are rapidly becoming obsolete. In this context, the demand for accurate, robust, and adaptive traffic classification under encryption and open-world conditions has grown more urgent than ever.

Traditional deep packet inspection methods, relying on port information [3] or plaintext payloads [4], are ineffective under dynamic port allocation and encrypted traffic [5]. In modern encrypted environments, payloads are inaccessible, ports are ambiguous, and traffic patterns are highly variable [1]. Existing machine learning [6], [7], [8] and deep learning [9] approaches based on flow-level statistics partially alleviate these challenges but depend on shallow or handcrafted features, limiting their discriminative power [10], [11]. Moreover, most studies assume a closed-world setting [6], [12], [13], [14], [15], [16], rendering them incapable of handling previously unseen traffic. Overall, these methods remain insufficient for real-world encrypted traffic classification. Through a comprehensive analysis of existing traffic classification systems and their limitations, we identify the following three key challenges in achieving accurate classification in open and encrypted environments:

Challenge 1: Labeled Data Scarcity, Class Imbalance, and Struggles with Maintaining Performance in Dynamic Networks. Real-world traffic data is inherently imbalanced—traffic is predominantly dominated by popular applications, while emerging or specialized services are underrepresented. Moreover, acquiring labeled data is costly, and the existing amount of labeled data is insufficient. Additionally, network anomalies such as packet loss, retransmissions, and delay fluctuations further complicate traffic patterns, posing greater challenges to model robustness. These factors degrade the accuracy and generalization ability of models, especially in dynamic environments.

Challenge 2: Weakly Discriminative Features Lead to Embedding Overlap, Limiting Classification Accuracy.

1556-6021 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:11:12 UTC from IEEE Xplore. Restrictions apply.

Existing neural network–based traffic classification methods [9], [13], [14], [17], [18], [19] typically train on a mixture of features—such as statistical descriptors and byte-level signals—but these features often face limitations in their discriminative power [20], [21]. Technically, modern encryption (e.g., TLS 1.3) renders payloads pseudo-random, masking traditional signatures. Furthermore, macroscopic statistical descriptors frequently overlap between functionally similar applications and are highly susceptible to environmental noise like network jitter. Consequently, the embeddings learned for each traffic class exhibit limited separability. Even when some classification models employ contrastive learning [22], [23] to better capture class characteristics, the low-discriminability features still lead to blurry inter-class boundaries. As a result, flows from the same class may vary widely in the feature space, while flows from different classes may be indistinguishable, thereby reducing classification accuracy.

Challenge 3: Inability to Discover and Classify Unknown Classes. Most prior works operate under closed-world assumptions, where all traffic classes are known a priori. In real-world scenarios, however, new applications and traffic types constantly emerge. Conventional classifiers struggle to recognize unknown classes, while existing open-set recognition methods often perform poorly on encrypted traffic due to feature ambiguity and dataset imbalance.

To address these challenges, we propose a unified Graph-based Contrastive Learning and Clustering (GCLC) framework for encrypted traffic classification in open-world environments.

Our core idea is to model encrypted traffic as traffic interaction graphs, capturing structural correlations between packets using only lightweight and basic packet-level features such as size, direction, and timestamp. We employ a Graph Neural Network (GNN) encoder to extract high-level representations from these graphs. Besides employing a supervised contrastive learning module to pull positive samples together and push negative samples apart, we also introduce a center-aware loss to further enhance feature discriminability and promote the formation of clear, compact class clusters. In addition, we introduce imbalanced samples to further enhance the model's robustness in handling dynamic environments and to mitigate data imbalance. Specifically, we propose a domain-specific graph augmentation strategy that generates new samples consistent with the characteristics of complex networks, thereby simulating traffic fluctuations in real-world networks. Finally, to support classification in open-world settings, we integrate a Mahalanobis distance-based new class detector along with an adaptive update mechanism, enabling the discovery and integration of new traffic classes.

Our approach offers the following key innovations:

1) Graph-based encrypted traffic modeling: Representing encrypted flows as traffic interaction graphs using simple packet-level features, enabling structural representation without relying on payload inspection or complex handcrafted features.

2) Traffic Graph Augmentation Strategy: Designing a graph-specific augmentation strategy tailored to network traffic, addressing issues of limited labeled data, class imbalance, and robustness in complex network environments by generating new samples aligned with real-world network characteristics.

3) Contrastive Clustering with Center-Aware Loss: Incorporating a residual projection head to preserve sample information, combining supervised contrastive learning with a center-aware objective to help the model learn highly discriminative features for forming well-defined and compact class clusters.

4) Open-World New Class Discovery: Integrating a Mahalanobis distance-based new class detector (for unknown new flows) and an adaptive model update strategy, enabling identification and integration of previously unseen traffic classes.

5) Comprehensive validation: Conducting extensive experiments on three public encrypted traffic datasets and one self-collected dataset. Experimental results show GCLC outperforms existing methods across four evaluation metrics, achieving an open-set discovery accuracy of 95.02%. The framework also demonstrates strong robustness under severe class imbalance and limited labeled data.

II. Related Work

Network traffic classification is critical for security monitoring, application identification, and network management. However, with the rise of encrypted protocols and increasingly complex traffic patterns, traditional methods have become less effective. This section reviews related work in three key areas: (i) conventional methods, (ii) machine learning-based approaches, and (iii) deep learning and graph-based methods.

Conventional Traffic Classification Methods: Early methods relied on static features such as IANA-assigned port numbers or protocol signatures. Port-based classification is now largely ineffective due to port reuse and dynamic allocation (e.g., HTTPS on port 443) [3]. Deep packet inspection (DPI) [4] offered higher accuracy by analyzing payloads, but it is limited against encrypted protocols (TLS, QUIC, VPN), incurs high computational costs, and raises privacy concerns. These limitations make traditional methods unsuitable for modern encrypted traffic.

Machine Learning-based Traffic Classification Methods: To overcome static rule-based approaches, machine learning (ML) techniques using flow-level statistical features have been explored. These include supervised methods (e.g., decision trees, SVM, random forests, KNN) and unsupervised methods like K-Means clustering [24], [25], [26], [27], [28], [29]. Studies like Williams and Zander [6] compared classifiers (e.g., C4.5, naïve Bayes) with various feature selection strategies, analyzing flow-level features such as packet counts and byte counts. Este et al. [7] applied SVMs to multi-class classification for TCP flows, while Alshamari and Zincir-Heywood [8] focused on encrypted VoIP traffic, finding decision tree models like C4.5 performed best for interpretability. Taylor et al. [12] proposed *AppScanner* for encrypted mobile apps, though its reliance on static feature templates limits adaptability to open-world scenarios.

Deep Learning and Graph-Based Traffic Classification Methods: Deep learning (DL) models have gained prominence for their ability to automatically learn hierarchical patterns.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:11:12 UTC from IEEE Xplore. Restrictions apply.

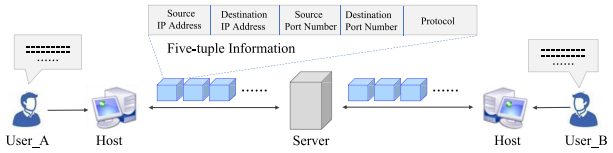


Fig. 1. The client–application traffic interaction process.

CNNs, LSTMs, and MLPs [9], [17], [18], [19] have been applied to raw packet sequences, eliminating manual feature engineering. While effective in closed settings, DL models require large labeled datasets and often fail to generalize to new classes.

Graph-Based Traffic Classification: Graph neural networks (GNNs) have recently been explored for their capacity to capture structural dependencies in traffic. Liu et al. [13] used RNN-based models for encrypted traffic classification, while Shen et al. [14] introduced *GraphDApp*, which converts traffic into interaction graphs for GNN-based application fingerprinting. Jiang et al. [15] proposed *FG-Net*, using a flow-relationship graph to improve fingerprint stability, but still relies on supervised labels. Yang et al. [16] integrated GNNs and Transformers in *MTSecurity* to detect encrypted threats, although it is limited to security-specific use cases.

Open-Set Recognition and Adaptive Methods: To address the emergence of unknown classes, open set recognition (OSR) has gained attention. Threshold-based detectors [5], semi-supervised temporal models [30], and masked autoencoder pretraining [10] have been proposed. Le et al. [11] introduced an adaptive model supporting incremental class updates. However, many of these rely on rigid feature templates, lack generalization, or require extensive tuning for deployment in dynamic environments.

Limitations of Existing Work: Despite advances in network traffic classification, existing methods still face significant limitations: real-world datasets are often sparse and imbalanced, making it difficult to train robust models; most approaches rely on closed-world assumptions, limiting their ability to identify new traffic classes; handcrafted or shallow features exhibit weak discriminative power in encrypted or obfuscated environments; unsupervised clustering tends to be fragmented or overlapping, reducing reliability; and incremental models incur high adaptation costs, with frequent updates and hyperparameter tuning constraining their deployment in dynamic network scenarios.

III. Problem Statement and Model Overview

A. Problem Statement

Encrypted traffic classification in open network environments presents significant challenges due to the dynamic and heterogeneous nature of real-world network conditions. This work aims to develop a robust framework capable of accurately classifying both known (labeled) and unknown (unlabeled or new) traffic types, while addressing practical constraints such as limited labeled data, class imbalance, and the presence of emerging applications.

In this context, network traffic is generated as sequences of packets that reflect application-specific communication behaviors. As illustrated in Fig. 1, we collect complete session-level traffic during client-application interactions. Each packet in a session contains essential metadata such as IP addresses, port numbers, and protocol identifiers, which are crucial for routing, parsing, and analysis.

To enable effective classification, we group packets into flows. A flow is defined as a sequence of packets sharing the same 5-tuple, including source/destination IP addresses, source/destination ports, and transport-layer protocol. Notably, we consider bidirectional flows—packets with reversed source and destination fields are treated as part of the same logical flow.

Definition 1 (Flow): Let S denote the set of all observed packets. A flow $F \subseteq S$ is a group of packets such that each packet $p \in F$ shares the same 5-tuple $(IP_s, IP_d, Port_s, Port_d, Protocol)$, where IP_s and IP_d represent the source and destination IP addresses, $Port_s$ and $Port_d$ represent the source and destination ports, and $Protocol$ is the transport protocol. If the source and destination address pair (i.e., $IP_s, IP_d, Port_s, Port_d$) of two packets is reversed, they are still considered to belong to the same flow. That is, when the source and destination fields are swapped, the two packets are still regarded as belonging to the same flow.

In an open environment, the goal of traffic classification is not only to recognize known categories but also to detect and handle unknown categories. During the training phase, the model learns discriminative representations for known categories; during the inference phase, the model determines whether a new flow belongs to a known category or represents a new category. The definition is as follows:

Definition 2 (Open Environment Traffic Classification): Let $\mathcal{C} = \{C_1, C_2, \ldots, C_K\}$ denote the set of known traffic categories, where each category C_i is associated with a labeled dataset $\mathcal{D}_i = \{(f_1, l_1), \ldots, (f_m, l_m)\}$, where $f_j \in \mathbb{R}^d$ is the feature vector and $l_j \in C_i$ is its label.

Assume the classifier M has been trained on these categories. For a new flow U, the model computes its feature representation $f_U = \Phi(U)$. To determine its class membership, the model calculates the distance between f_U and the center μ_i of each known class. If the distance $d(f_U, \mu_i)$ is below a predefined threshold ϵ, U is assigned to category C_i; otherwise, it is considered a candidate for a new class.

This paper adopts the Mahalanobis distance [31] as the similarity metric $d(f_U, \mu_i) = \sqrt{(f_U - \mu_i)^\top \Sigma_i^{-1} (f_U - \mu_i)}$, where μ_i and Σ_i denote the mean vector and covariance matrix of category C_i, respectively. In the context of our proposed GCLC framework, this feature representation f_U corresponds precisely to the projected graph embedding z' extracted by the GNN encoder, which will be detailed in subsequent sections.

B. Model Overview

To address the challenges of encrypted traffic classification in open environments, we propose a novel framework named **Graph-based Contrastive Learning Clustering (GCLC)**. This framework is designed to simultaneously classify known traffic types and discover new categories adaptively. It consists

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:11:12 UTC from IEEE Xplore. Restrictions apply.

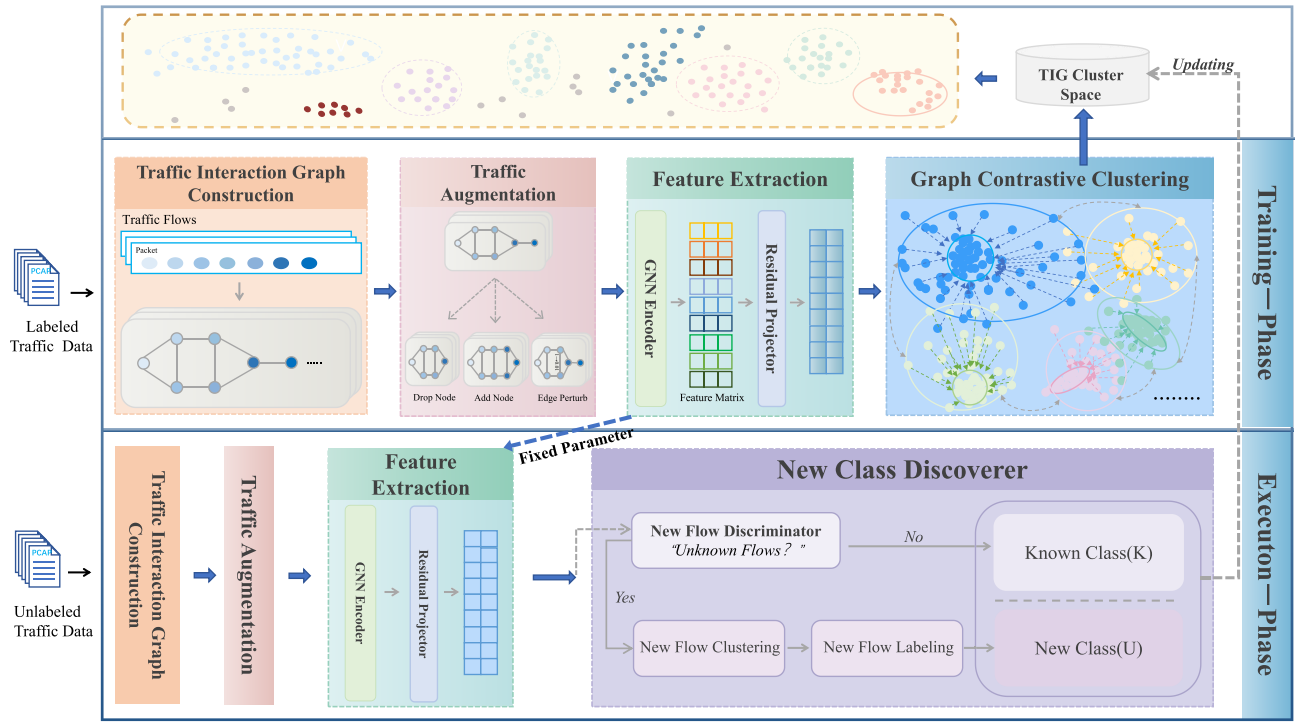


Fig. 2. Overview of the proposed GCLC framework. During training, labeled traffic graphs are used in supervised contrastive clustering to form distinct class clusters. During execution, new flows are embedded and compared against known clusters for classification or new class detection.

of two main stages: a *training-phase* and an *execution-phase*, as illustrated in Fig. 2.

Training Phase Modules:

(1) Traffic Interaction Graph Construction: Packet sequences are aggregated into flows and transformed into graph representations using basic packet-level features. This approach captures application-specific interaction patterns without relying on complex manual features.

(2) Traffic Augmentation: To address the issues of data scarcity, class imbalance, and struggles with maintaining performance in dynamic networks in complex network environments, we propose a graph-based data augmentation strategy. This strategy generates additional samples that align with network characteristics by simulating variations in network behaviors, thereby enhancing the model's robustness and generalization ability.

(3) Feature Extraction: A Graph Neural Network (GNN) encoder is trained to learn graph-level embeddings. The encoder leverages neighborhood aggregation and shallow architecture to efficiently capture discriminative features.

(4) Graph Contrastive Clustering: In addition to using supervised contrastive learning to encourage embeddings of the same class to cluster tightly while maximizing inter-class separation, we also introduce a center-aware loss to make the boundaries of difficult class clusters clearer and improve cluster compactness.

Execution Phase Modules:

(1) Traffic Interaction Graph Construction: New flows are processed into graph form using the same procedure as in training.

(2) Traffic Augmentation: The proposed graph-based data augmentation strategy is used to simulate variations in network behaviors, generating samples that align with network characteristics to construct contrastive learning positive and negative sample pairs, addressing the issue of limited data.

(3) Feature Extraction: The pretrained GNN encoder (with frozen parameters) extracts embeddings for incoming flows.

(4) New Class Discoverer: By determining an appropriate threshold based on class-conditional Mahalanobis distances, the distance between each flow embedding and the centers of existing class clusters is compared against the threshold to decide whether the flow belongs to a new class.

IV. Methodology

This section presents the detailed design of the proposed framework, encompassing the construction of traffic interaction graphs, the graph augmentation strategy, the design of the graph-based feature extractor, the supervised contrastive clustering mechanism, and the process for discovering new traffic categories.

A. Traffic Interaction Graph Construction

Modeling encrypted traffic as graphs offers significant advantages in capturing both structural and temporal dependencies inherent in network flows. Such representations are particularly suitable for variable-length traffic sequences and enable structure-aware learning via Graph Neural Networks (GNNs). This graph-based abstraction enhances the model's capability to discriminate between traffic categories based on subtle behavioral patterns and communication dynamics.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:11:12 UTC from IEEE Xplore. Restrictions apply.

Due to the heterogeneity of application-layer protocols and communication behaviors, traffic flows inherently exhibit diverse structural patterns. To effectively capture these variations, we construct a *Traffic Interaction Graph (TIG)* for each flow, based on lightweight packet-level attributes: packet size, direction and inter-arrival time.

As illustrated in Fig. 3, each directed arrow denotes a packet transmission between the client and server. The numeric value indicates the packet size, with its sign representing direction: positive values (in orange) denote upstream packets from client to server, while negative values (in blue) denote downstream packets from server to client.

To retain both temporal correlations and structural semantics, we adopt a graph representation strategy inspired by [32], forming a unified and expressive representation for downstream tasks. The construction of the TIG consists of the following three components:

Vertices: Each node in the TIG corresponds to a packet in the flow. Nodes are annotated with the signed packet size and indexed by their sequence order. Packet direction is encoded by node color (orange for upstream, blue for downstream), and packets with the same direction are grouped into distinct layers to capture directional communication patterns. Within each layer, packets are temporally ordered and connected accordingly.

Edges: Edges in the TIG are categorized into two types:

Intra-layer edges: These edges connect temporally adjacent packets within the same direction (layer), preserving fine-grained temporal ordering and local dependencies.

Inter-layer edges: These edges encode transitions between communication directions and are constructed using the following rules: The first node in each layer is connected to the first node in the subsequent layer.The last node in each layer is connected to the last node in the subsequent layer.This design captures the bidirectional nature of application-level interactions.

Edge Weights: To incorporate temporal dynamics, edge weights are assigned based on inter-arrival times between connected packets. Recognizing that network-induced noise (e.g., jitter, delay) may introduce variability, we discretize inter-arrival time into two coarse-grained categories: **Short-term:** Intervals less than or equal to 1 second are encoded with a weight of 1. **Long-term:** Intervals greater than 1 second are encoded with a weight of 0.01. This empirical constant mitigates network noise like collisions. During GNN aggregation, this near-zero weight acts as a soft-pruning mechanism, attenuating message passing across arbitrary delays to focus on semantically meaningful packet bursts.

This coarse-grained time encoding improves robustness to temporal noise while preserving macro-level timing characteristics.

Example Construction: Fig. 4 shows the resulting TIG corresponding to the example flow in Fig. 3, with a packet size sequence [64, −184, 484, −64, −64, 160, 118, −164, 652]. The vertex set is initialized as V = [64, −184, 484, −64, −64, 160, 118, −164, 652], and edge weights are determined by inter-arrival intervals E = [1, 1, 1, 1, 1, 1, 1, 1, 0.01]. Intra-layer and inter-layer edges are established iteratively according to the rules above. The final graph $G = (V, E)$ represents the desired Traffic Interaction Graph.

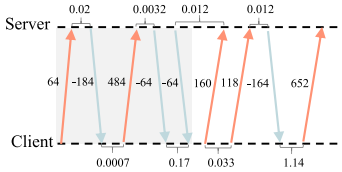


Fig. 3. Packet-level client-server interaction for flows.

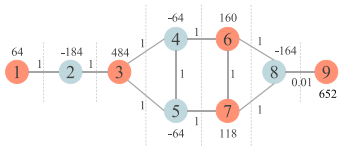


Fig. 4. Traffic interaction graph (TIG) for flows.

B. Graph Augmentation Strategy for Traffic With Insufficient Labeled Data, Class Imbalance, and Robustness

Real-world traffic datasets often face issues of limited labeled data and class imbalance, with common applications dominating the traffic, while niche or emerging applications are underrepresented. This imbalance causes models to rely heavily on features from the common classes, neglecting the details of the minority classes, leading to poor performance on minority traffic and reduced accuracy. Additionally, due to the high cost of acquiring labeled data, the existing labeled data is insufficient, which lowers the model's generalization ability.

Network anomalies such as packet loss, retransmissions, and fluctuating delays further complicate traffic patterns, challenging model robustness, especially when labeled data is scarce.

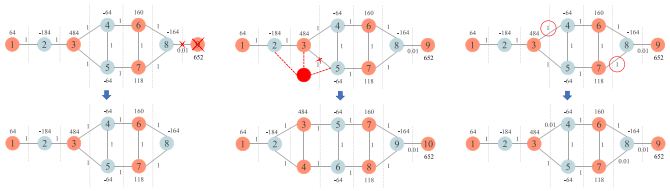
To address these challenges, we propose a domain-specific graph augmentation strategy tailored for TIGs. This method aims to: (i) mitigate class imbalance by synthetically enriching minority class samples, (ii) enhance robustness to network-induced anomalies, and (iii) preserve the semantic integrity of encrypted traffic behavior.

Let the original dataset be defined as $\mathcal{D} = \{(G_i, y_i) \mid i = 1, \ldots, N_{total}\}$, where G_i is a TIG sample and $y_i \in \{1, 2, \ldots, K\}$ is its class label. For a given class c, let $\mathcal{D}_c = \{G_i \mid y_i = c\}$, and $|\mathcal{D}_c| = n_c$. We define a set of augmentation operations as $\mathcal{A} = \{\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3\}$. where: $\mathcal{A}_1$: Random Node Addition (simulating packet retransmission). $\mathcal{A}_2$: Random Node Dropping (simulating packet loss).$\mathcal{A}_3$: Edge Attribute Perturbation (simulating variable delay).

For any graph G, an augmentation operation is selected uniformly at random $\hat{G} = \mathcal{A}_k(G), \quad k \sim \text{Uniform}\{1, 2, 3\}$. Let $n_{\max} = \max_c \frac{n_c}{2}$ denote the target number of samples per class. Each class c is augmented until $|\mathcal{D}'_c| = n_{\max}$. The final balanced dataset is $\mathcal{D}' = \bigcup_{c=1}^{K} \mathcal{D}'_c$.

In practical network environments, events such as packet loss, retransmission, and delay jitter occur infrequently. Therefore, augmentation operations are applied with low probability, simulating realistic noise patterns while preserving the overall

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:11:12 UTC from IEEE Xplore. Restrictions apply.



(a) Random node dropping (b) Random node addition (c) Edge attribute perturbation

Fig. 5. Illustration of various network traffic perturbations: (a) Random node dropping, (b) Random node addition, and (c) Edge attribute perturbation.

semantics of traffic interaction. We now describe each augmentation operation in detail.

Random Node Dropping: To emulate packet loss, a subset of nodes is randomly removed from the TIG. Associated edges are also deleted, and the graph is reindexed to preserve consistency. This simulates scenarios where packets are dropped due to congestion or transmission errors. As encrypted traffic often tolerates minor packet loss without losing its semantic structure, this operation preserves essential flow characteristics. As shown in Fig. 5a, node 9 is removed to simulate packet loss.

Random Node Addition: This operation simulates packet retransmission, which commonly occurs in response to network errors. Within each directional layer of a TIG (i.e., consecutive upstream or downstream packets), a new node is inserted after the last packet. The inserted node inherits the size and direction of its predecessor and is connected to maintain temporal ordering. As shown in Fig. 5b, a duplicate packet is inserted to simulate retransmission.

Edge Attribute Perturbation: To simulate temporal variability (e.g., RTT fluctuations), we randomly perturb a subset of edge weights in the TIG. Specifically, edge weights are toggled between $1 \leftrightarrow 0.01$, modeling transitions between short-term and long-term delays. The selection follows an i.i.d. uniform distribution across both intra- and inter-directional edges. As illustrated in Fig. 5c, we simulate delay variation by modifying edge weights.

These augmentation operations collectively enhance the model's ability to generalize across imbalanced, noisy, and evolving traffic environments.

C. Feature Extraction

To facilitate effective representation learning over graph-structured traffic data, we propose a feature extraction module that integrates multi-layer graph convolution with residual projection. The objective is to capture both local and global structural patterns within flows, enhance the semantic expressiveness of Traffic Interaction Graphs (TIGs), and produce high-quality embeddings for downstream classification and clustering tasks.

The module consists of three main components: (a) multi-layer GraphSN-based graph encoding, (b) hierarchical feature fusion, and (c) a residual projection head.

Graph Feature Encoding via GraphSN:

In the initial stage of feature extraction, we utilize multiple stacked GraphSN layers to perform node-level message passing and representation learning. Each GraphSN unit incorporates a learnable adjacency adaptation mechanism, allowing dynamic adjustment of the graph structure to suppress over-smoothing while preserving meaningful node-level distinctions. Specifically, the adjusted adjacency matrix $\tilde{\mathbf{A}}$ is computed as $\tilde{\mathbf{A}} = (1 - \mathbf{I}) \cdot \mathbf{A} + \mathbf{I} \cdot \mathrm{diag}(e) \cdot \mathrm{diag}(\mathbf{A})$, where $\mathbf{A} \in \mathbb{R}^{N_v \times N_v}$ is the original adjacency matrix, $\mathbf{I} \in \mathbb{R}^{N_v \times N_v}$ is the identity matrix, and $e \in \mathbb{R}^{N_v}$ is a learnable parameter vector that controls the scaling of self-loops. This formulation enables the model to regulate the influence of each node's own features during propagation.

Node embeddings are then updated as $X' = \mathrm{ReLU}\left(\mathrm{Linear}\left(\mathrm{MLP}(\tilde{\mathbf{A}}X)\right)\right)$, where $X \in \mathbb{R}^{N_v \times d}$ is the input node feature matrix, $X' \in \mathbb{R}^{N_v \times d'}$ is the updated node feature matrix, and d and d' are the input and hidden dimensions, respectively. MLP denotes a multi-layer perceptron, and ReLU is the non-linear activation function. Batch normalization is applied between layers to stabilize training and improve convergence.

Multi-Scale Feature Fusion:

To preserve information at multiple levels of abstraction, we retain the intermediate outputs from each GraphSN layer. At the end of the encoding phase, we concatenate the outputs of all layers, along with the original input features, to form a multi-scale representation. Let $X^{(l)} \in \mathbb{R}^{N_v \times d'}$ represent the output of the l-th GraphSN layer. The multi-scale concatenation is $H = \mathrm{Concat}(X^{(0)}, X^{(1)}, \ldots, X^{(L)}) \in \mathbb{R}^{N_v \times d' \cdot (L+1)}$, where L is the total number of GraphSN layers, and d' is the number of hidden units. This fusion strategy ensures that both fine-grained local structures and high-level global semantics are captured.

The concatenated node embeddings are aggregated along the node dimension using a readout operation (e.g., summation or mean pooling) to derive a fixed-size graph-level representation vector $z \in \mathbb{R}^{d' \cdot (L+1)}$, This aggregated vector z is used as the input to the residual projection head.

Residual Projection Head:

To project the high-dimensional representation z into a lower-dimensional embedding space suitable for contrastive learning or clustering, we introduce a Residual Projector module. This component follows a dual-path design: The **main path** applies two fully connected layers interleaved with batch normalization and ReLU activation, focusing on extracting nonlinear transformations. The **residual shortcut** directly propagates the original input z through a linear projection.The final projected embedding is computed as $z' = \mathrm{MainPath}(z) + \mathrm{Shortcut}(z)$, where $z \in \mathbb{R}^{d' \cdot (L+1)}$ is the high-dimensional feature vector and $z' \in \mathbb{R}^{d_{proj}}$ is the final projected embedding, with d_{proj} being the projection dimension. This design preserves the semantic continuity of the original features while enhancing the learning of task-relevant representations.

Augmented View Encoding for Contrastive Learning: To further enhance robustness and generalization, we integrate graph augmentation directly into the feature extraction pipeline. Specifically, two independent augmentations are applied to the input TIG, where each augmentation is randomly selected from the predefined set $\mathcal{A} = \{\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3\}$. This results in two structurally perturbed but semantically consistent graph views.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:11:12 UTC from IEEE Xplore. Restrictions apply.

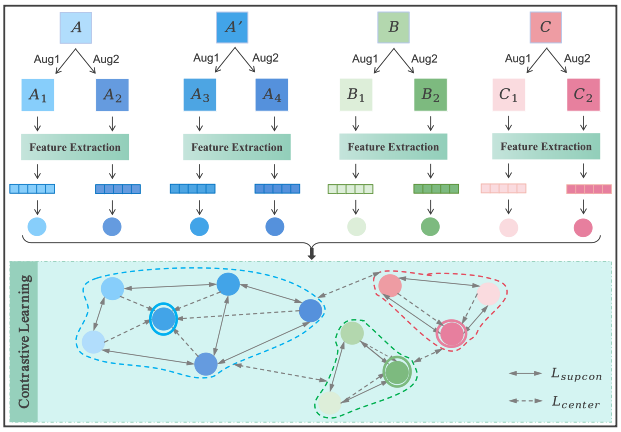


Fig. 6. Illustration of the dual-branch loss: $\mathcal{L}_{\text{supcon}}$ pulls samples of the same class closer and pushes those of different classes apart, while $\mathcal{L}_{\text{center}}$ encourages clustering around class centers and separates different class clusters.

These augmented views are processed by a shared encoder, producing embeddings z_1' and z_2', which form a positive pair in the contrastive learning framework. This mechanism encourages the model to learn invariant representations under structural perturbations, facilitating improved intra-class compactness and inter-class separation.

To obtain negative pairs, we adopt a supervised contrastive learning strategy: Views from the same class (original and augmented) are treated as positive pairs; Views from different classes are treated as negative pairs.

By jointly optimizing both positive and negative objectives, the model learns to distinguish semantically meaningful variations while remaining resilient to structural noise. This ultimately contributes to enhanced classification performance, especially in open and dynamic network environments.

D. Graph Contrastive Learning Clustering

Traffic flows from the same application class often exhibit latent commonalities, while flows from different classes diverge significantly due to variations in protocol stacks, service behaviors, and communication semantics. Our goal is to cluster traffic interaction graphs in the embedding space such that traffic samples from the same class are tightly clustered, while samples from different classes are well separated.

To achieve this goal, we employ a dual-branch contrastive learning framework to enhance class discriminability. Unlike traditional methods that rely on simple supervised contrastive learning, we introduce a center-aware loss to form internally compact, clearly separated, and non-confused class clusters. While traditional supervised contrastive learning can bring similar traffic samples closer and push dissimilar ones apart, it struggles when faced with classes that have subtle feature similarities. To address this issue, we incorporate a center-aware loss to improve the separation of hard-to-distinguish clusters, making the overall clustering more compact and ensuring each class forms a distinct cluster. Fig. 6 illustrates the effects of these two loss functions.

The two branches share a common encoder and projection head. Each graph sample is projected into the embedding space, where class cluster centers are also learned. The training objective jointly optimizes the supervised contrastive loss and center-aware loss (composed of intra-class aggregation, inter-class separation, and nearest cluster classification), ensuring that similar samples are grouped together in the feature space, different class clusters are adequately separated, and classification is achieved using the "sample-nearest cluster" decision.

Supervised Contrastive Loss:

Given the two augmented views per graph produced by the feature-extraction module (for each original graph G_i we obtain $\tilde{G}_i^{(1)}$ and $\tilde{G}_i^{(2)}$), we encode them with shared weights to obtain two embeddings per sample. Denote the (optionally ℓ_2-normalized) embeddings by $\hat{\mathbf{z}}_u' = \mathbf{z}_u'/\|\mathbf{z}_u'\|_2$. For a minibatch of B original graphs, we thus have $2B$ views; index them by $u \in \{1, \ldots, 2B\}$ with class label y_u.

We adopt the supervised contrastive objective (one anchor per view). Define the positive set for anchor u as $\mathcal{P}(u) = \{v \in \{1, \ldots, 2B\} \setminus \{u\} \mid y_v = y_u\}$, and define $\mathcal{N}(u) = \{1, \ldots, 2B\} \setminus \{u\}$. The (temperature-scaled) cosine similarity is given by $s_{uv} = \frac{\hat{\mathbf{z}}_u'^{\top} \hat{\mathbf{z}}_v'}{\tau}$. The per-anchor loss and the batch loss are $\ell_u^{\text{sup}} = -\frac{1}{|\mathcal{P}(u)|} \sum_{v \in \mathcal{P}(u)} \log \frac{\exp(s_{uv})}{\sum_{a \in \mathcal{N}(u)} \exp(s_{ua})}$, $\mathcal{L}_{\text{supcon}} = \frac{1}{2B} \sum_{u=1}^{2B} \ell_u^{\text{sup}}$. This pulls embeddings of the same class (including cross-view pairs of the same sample/class) together while pushing away embeddings from different classes.

Center-aware Loss:

In addition to using supervised contrastive learning to bring similar samples closer and push dissimilar ones apart, we also introduce a center-aware loss to enhance cluster separation, ensuring each class is compact and distinct. $\mathcal{L}_{\text{center}} = \lambda_{\text{intra}} \mathcal{L}_{\text{intra}} + \lambda_{\text{inter}} \mathcal{L}_{\text{inter}} + \lambda_{\text{cls}} \mathcal{L}_{\text{cls}}$, where the three terms respectively promote (i) $\mathcal{L}_{\text{intra}}$: intra-class compactness, (ii)$\mathcal{L}_{\text{inter}}$: inter-class separation with adaptive margins, and (iii)$\mathcal{L}_{\text{cls}}$:sample-to-own-center nearest-neighbor consistency.

(i) Intra-class Compactness. Let $\mathcal{K}$ be the set of classes present in the current training epoch, and $S_k = \{u : y_u = k\}$ the index set of samples (across both views) of class k, with $S = \bigcup_{k \in \mathcal{K}} S_k$ and $|S| = 2B$. For each class k, denote its center by $\mathbf{c}_k$.

We define normalized inverse-distance weights as $w_u = \frac{\frac{1}{d(\mathbf{z}_u', \mathbf{c}_k) + \epsilon}}{\sum_{v \in S_k} \frac{1}{d(\mathbf{z}_v', \mathbf{c}_k) + \epsilon}}$, $u \in S_k$. We penalize the (weighted) squared deviation of samples from their class center:

$$\mathcal{L}_{\text{intra}} = \frac{1}{|S|} \sum_{u \in S} w_u \left(\alpha \cdot d(\mathbf{z}_u', \mathbf{c}_{y_u})\right)^2, \tag{1}$$

where w_u assigns larger weights to samples closer to the class center, allowing prototype updates to be guided by more representative instances while reducing the influence of noisy or outlying samples.

(ii) Inter-class Separation. Let $d_{ij} = d(\mathbf{c}_i, \mathbf{c}_j)$ be the center distance for classes $i \neq j$. We adapt pairwise margins by class difficulty. Define the difficulty of class k by

$$d_k = \frac{\frac{1}{|S_k|} \sum_{u \in S_k} d(\mathbf{z}_u', \mathbf{c}_k)}{\frac{1}{|\mathcal{K} \setminus \{k\}|} \sum_{j \in \mathcal{K} \setminus \{k\}} d(\mathbf{c}_k, \mathbf{c}_j)}. \tag{2}$$

Let $\rho \in (0, 1)$ (e.g., $\rho = 0.7$) and define the "hard-class" set $\mathcal{H}$ as the top-ρ fraction of classes with the largest d_k. Set the pair-type scaling:

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:11:12 UTC from IEEE Xplore. Restrictions apply.

$$s_{ij} = \begin{cases} 1.5, & i \in \mathcal{H},\ j \in \mathcal{H}, \\ 1.3, & \text{exactly one of } i, j \in \mathcal{H}, \\ 1.0, & \text{otherwise}, \end{cases} \qquad m_{ij} = m_0 \cdot s_{ij},$$

with base margin $m_0 > 0$. We use a non-linear margin penalty

$$\varphi(d_{ij}, m_{ij}) = \begin{cases} 5\left(\dfrac{m_{ij} - d_{ij}}{m_{ij}}\right)^2, & d_{ij} < m_{ij}, \\ \dfrac{1}{2}\exp\left(-\dfrac{d_{ij} - m_{ij}}{2}\right), & d_{ij} \geq m_{ij}, \end{cases}$$

and average it over unordered pairs $\mathcal{P} = \{(i, j) \in \mathcal{K} \times \mathcal{K} : i < j\}$:$\mathcal{L}_{\text{inter}} = \frac{1}{|\mathcal{P}|}\sum_{(i,j)\in\mathcal{P}} \varphi(d_{ij}, m_{ij})$.

(iii) Nearest Neighbor Margin Loss. For each sample u, define $d_u^+ = d(\mathbf{z}'_u, \mathbf{c}_{y_u})$ and $d_u^- = \min_{j\in\mathcal{K}\setminus\{y_u\}} d(\mathbf{z}'_u, \mathbf{c}_j)$. We enforce that u is closer to its own center than to any other center by a margin $\gamma > 0$: $\mathcal{L}_{\text{cls}} = \frac{1}{|S|}\sum_{u\in S} w_u\,[\,d_u^+ - d_u^- + \gamma\,]_+$, $[x]_+ = \max(x, 0)$.

Weighting by w_u emphasizes boundary and outlying samples.

Final Objective and Prediction:

The total training objective combines supervised contrast and center-aware regularization, $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{supcon}} + \eta\,\mathcal{L}_{\text{center}}$ with trade-off $\eta > 0$. At inference, we assign labels by nearest center, $\hat{y} = \arg\min_{k\in\mathcal{K}} d(\mathbf{z}', \mathbf{c}_k)$, which is consistent with the training-time nearest-center margin term $\mathcal{L}_{\text{cls}}$. During training, we select the sample within each class that has the minimum distance to all other samples as the class center, $u^\star = \arg\min_{u\in S_k}\sum_{v\in S_k} d(\mathbf{z}'_u, \mathbf{z}'_v)$, $\mathbf{c}_k \leftarrow \mathbf{z}'_{u^\star}$.

At the end of each training epoch, we perform a momentum update: $\mathbf{c}_k \leftarrow m\cdot\mathbf{c}_k + (1-m)\cdot\tilde{\mathbf{c}}_k$, where $\tilde{\mathbf{c}}_k = \sum_{u\in S_k} w_u\,\mathbf{z}'_u$. We set $m = 0.9$ to provide temporal smoothing, which mitigates the "moving target" problem and ensures stable convergence by acting as a consistent anchor in the feature space.

E. New Class Discoverer

This section presents a new-class discovery method that identifies and clusters unknown classes by combining Mahalanobis distance–based new class detection with eigengap-driven adaptive spectral clustering. First, new samples are detected via Mahalanobis distance, with a global threshold determined via a hold-out validation strategy to prevent data leakage. Next, spectral clustering guided by the eigengap heuristic groups the new samples. After discovering new classes, the model is incrementally updated—covering label assignment, model extension, and contrastive loss updates—enabling dynamic adaptation to new classes and improved classification performance.

Mahalanobis Distance-Based New Class Detection: To detect new classes, we model the distribution of each known class k using its mean $\mu_k = \frac{1}{n_k}\sum_{i=1}^{n_k} z_i^{'(k)}$ and covariance $\Sigma_k = \frac{1}{n_k-1}\sum_{i=1}^{n_k}(z_i^{'(k)} - \mu_k)(z_i^{'(k)} - \mu_k)^T + \beta\mathbf{I}$, and compute the Mahalanobis distance for a test sample z', $d_M(z', k) = \sqrt{(z'-\mu_k)^T\Sigma_k^{-1}(z'-\mu_k)}$. Here, we empirically set the regularization parameter β to 10^{-6}. This small constant acts as a numerical stabilizer, ensuring that the high-dimensional covariance matrix remains strictly positive-definite and invertible without distorting the underlying feature distributions. This distance accounts for feature correlations and is more robust than Euclidean distance in high-dimensional spaces.

To determine the global threshold θ_{global} without accessing real novel classes (which would constitute data leakage), we employ a hold-out validation strategy. We partition the known training classes into a fitted subset and a hold-out subset (acting as pseudo-unknown classes). By plotting the ROC curve on this validation set and maximizing the Youden index, we obtain an optimal distance cut-off. Since absolute distance values vary with model capacity and class count, we map this cut-off to its corresponding cumulative percentile P_{opt} within the fitted classes' distance distribution. Finally, we apply this scale-invariant percentile P_{opt} to the distance distribution of the final trained model to set the operational threshold θ_{global}. A test sample is considered new if $\min_k d_M(z', k) > \theta_{\text{global}}$.

Adaptive Clustering via Spectral Eigengap:

For the new sample set S_{new}, we perform spectral clustering to discover latent classes. A similarity matrix W is built using the RBF kernel, $W_{ij} = \exp\left(\frac{-|z'_i - z'_j|^2}{2\sigma^2}\right)$, where $\sigma = \text{median}\left(d(z'_i, z'_j) : i < j,\ z'_i, z'_j \in S_{\text{new}}\right)$. The normalized Laplacian is defined as $\mathbf{L}_{\text{norm}} = \mathbf{I} - \mathbf{D}^{-1/2}\mathbf{W}\mathbf{D}^{-1/2}$. We then compute the eigenvalues $\lambda_1 \leq \ldots \leq \lambda_m$, and determine the optimal number of clusters $K^*_{new} = \arg\max_{K_{new}}(\lambda_{K_{new}+1} - \lambda_{K_{new}})$ based on the eigengap heuristic. This identifies the most significant separation in the spectral domain, after which we apply K-means with K^*_{new} to cluster the new samples.

Incremental Model Update:

Once new clusters are discovered, the model is updated as follows: Each new cluster $C_{k_{new}}$ is assigned a temporary mathematical label $K + k_{new}$, where K is the number of known classes. While the system autonomously isolates these novel clusters, assigning their actual semantic meanings requires a periodic human-in-the-loop review. A new encoder is initialized with shared parameters and an updated output layer. New samples are integrated into the supervised contrastive loss using updated class labels. New class centers are computed using local density weights, $\mu_{k_{new}} = \frac{\sum_{i:y_i=k_{new}}\rho_i z_i}{\sum_{i:y_i=k_{new}}\rho_i}$. A test sample is then classified by its closest updated center, $\hat{y} = \arg\min_{k=1,\ldots,K+K^*_{new}} \|z - \mu_k\|$. Using the updated encoder to reassign class labels, recalculating the mean value μ_K of each class, and updating the similarity matrix and covariance matrix parameters.

V. Experiments

We conduct a series of experiments to evaluate the performance of the proposed GCLC framework, aiming to address the following key questions: 1) How does GCLC perform compared with other methods in the closed-world classification task? 2) Can GCLC effectively identify new classes of traffic in the open-world classification task? 3) Can GCLC enable traffic clusters of each class to have clear boundaries in the feature space? 4) What is the convergence speed and stability of GCLC? 5) How does the choice of threshold affect the performance of new traffic class identification? 6) Can GCLC meet the low-latency inference requirements for real-world deployment? 7) What are the contributions of each key component to the overall performance?

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:11:12 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SUMMARY OF BASELINE METHODS FOR ENCRYPTED TRAFFIC CLASSIFICATION

Model	Year	Input Features	Learning Paradigm	Core Design Idea	Unknown Flow Recognition
AppScanner [12]	2016	handcrafted Statistical features from TCP flows	Supervised learning on traditional classes	Uses self-collected data for mobile app detection under strict conditions	✗
FS-Net [13]	2019	Flow sequences with direction encoding	Seq2Seq + reconstruction-based learning	Reconstructs sequence-level features for better flow semantics	✗
FlowPrint [30]	2020	Domain-specific features with temporal correlation	Semi-supervised learning with labeled anchors	Uses behavioral fingerprinting based on time-location consistency	✓
GraphDApp [14]	2021	Traffic Interaction Graph(TIG)	GNN-based supervised training	Captures flow directionality and structure-aware embeddings	✗
FG-Net [15]	2022	Flow-Relationship Graph(FRG)	Supervised GNN classification	Encodes flow-level hierarchy for anomaly and app detection	✗
YaTC [10]	2023	Multi-layer flow visualization with Transformer	Transformer + MAE pretraining + fine-tuning	Learns flow patterns adaptively with cross-view supervision	✓
Velatin [11]	2024	Deep features + adaptive classifier	Supervised + incremental adaptation	Learns class boundaries adaptively in open environments	✓

A. Experiment Settings

Datasets: To comprehensively evaluate our method, we conduct experiments on four real-world encrypted traffic datasets that span a wide range of applications, network conditions, and traffic behaviors:

APPLICATIONS_21 (Self-Collected): This dataset consists of TLS-encrypted traffic from 21 popular applications. To capture real-world dynamics, data was collected via Wireshark on Windows 10/11 laptops and desktops from Sept. to Nov. 2023, spanning residential Wi-Fi, campus LAN (peak/off-peak), and 4G/5G hotspots. We use 17,203 flows for training and 4,301 flows for testing.

ISCX-VPN Dataset [33]: A dataset designed for VPN traffic analysis. We extract 13 VPN-related application classes and use 2,660 flows for training and 664 flows for testing.

USTC-TFC Dataset [34]: Comprising traffic from 10 benign applications and 10 malware families. One class is excluded due to extreme imbalance, resulting in 19 active classes. The training and test sets contain 15,667 and 3,741 flows, respectively.

VNAT Dataset [35]: A VPN/Non-VPN dataset with traffic from 10 applications across five functional categories: Streaming, VoIP, Chat, Command & Control, and File Transfer. We use 4,094 flows for training and 1,021 flows for testing.

Baselines: We compare GCLC against seven representative methods, a comparative summary of these baselines is shown in Table I.

The selected baselines cover a variety of classification paradigms, including classical, sequence-based, graph-based, and transformer-based methods. Notably, only models such as FlowPrint [30] and Velatin [11] explicitly support open-world scenarios. Therefore, GCLC is compared with these two models specifically in terms of new traffic identification performance.

Evaluation Metrics: To evaluate classification performance, we adopt four standard metrics: accuracy (AC), precision (PR), recall (RC), and F1 score. These metrics are defined as follows: Accuracy = $\frac{TP+TN}{TP+TN+FP+FN}$, Precision = $\frac{TP}{TP+FP}$, Recall = $\frac{TP}{TP+FN}$, $F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision}+\text{Recall}}$, where: **TP =** true positives, **TN =** true negatives, **FP =** false positives, **FN =** false negatives.

Implementation Details: All experiments were conducted using PyTorch 2.4.1 with CUDA acceleration. The model was trained on a workstation equipped with an NVIDIA GeForce RTX 4090 GPU.

During training, we used a batch size of 128 and employed the Adam optimizer with an initial learning rate of 2.5×10^{-3}. A learning rate decay factor of 0.5 was applied when validation performance plateaued. Weight decay (i.e., L_2 regularization) was set to 2×10^{-3} to prevent overfitting.

The graph augmentation hyperparameters were empirically selected based on validation performance: Node dropping ratio: 0.05, Edge perturbation ratio: 0.10, Node addition ratio: 0.05. These augmentations specifically target minority classes to mitigate imbalance. The 0.05 ratio physically simulates a realistic 5% packet loss and retransmission rate, introducing robust network noise without destroying core structural semantics. Unless otherwise specified, all experiments adhere to this configuration. Each experiment was repeated three times with different random seeds, and the average performance is reported.

B. Closed-World Classification Performance

In closed-world classification tasks, GCLC achieves the best overall performance and obtains the highest scores on most metrics, demonstrating excellent and stable performance. As shown in Table II, GCLC achieves an accuracy of 96.84% on APPLICATIONS21, outperforming the second-best FG-Net (93.67%) by 3.17%. On the ISCX-VPN dataset, its F1-score reaches 98.08%, exceeding the runner-up by 2.46%. This advantage mainly stems from the graph encoder's efficient modeling of structural and temporal features, along

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:11:12 UTC from IEEE Xplore. Restrictions apply.

TABLE II
PERFORMANCE COMPARISON ACROSS MULTIPLE DATASETS AND METHODS

Method	APPLICATIONS21				ISCX-VPN				USTC-TFC				VNAT			
	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1	AC	PR	RC	F1
AppScanner [12]	90.33	89.23	91.96	90.57	78.62	78.32	79.19	78.75	63.71	61.09	64.11	62.37	88.62	87.91	90.10	88.26
FS-Net [13]	74.89	73.72	74.74	74.23	82.94	81.97	83.12	82.54	70.19	68.10	71.92	69.13	85.10	83.84	84.36	84.47
FlowPrint [30]	61.43	59.29	61.23	60.24	70.21	69.13	70.95	70.03	58.03	55.30	56.21	56.63	71.37	69.84	72.07	70.60
GraphDApp [14]	79.14	78.07	79.13	78.60	81.33	79.96	80.92	80.44	79.02	78.17	78.91	78.59	85.13	83.30	84.92	84.21
FG-Net [15]	93.67	92.63	91.26	91.94	96.78	95.08	96.17	95.62	95.33	93.39	95.83	94.35	98.77	**96.80**	98.94	**97.78**
YaTC [10]	89.12	87.82	89.83	88.81	83.28	81.29	82.39	81.84	80.71	78.70	78.62	79.69	85.29	82.19	82.35	83.71
Velatin [11]	85.67	84.38	86.12	85.24	91.67	90.94	92.16	91.55	85.33	85.78	86.04	85.55	94.20	93.60	93.69	93.90
Ours	**96.84**	**93.12**	**97.13**	**95.08**	**98.16**	**97.73**	**98.44**	**98.08**	**96.54**	**94.18**	**98.09**	**95.35**	**98.83**	96.47	**99.13**	97.64

TABLE III
NEW CLASS DETECTION ON APPLICATIONS_21

Method	ACC	PR	Recall	F1
FlowPrint	57.43	56.14	55.84	55.99
Velatin	83.67	81.97	82.23	82.10
Ours	**95.02**	**94.71**	**96.45**	**95.57**

TABLE IV
CLUSTERING METRICS COMPARISON

Method	Clustering Accuracy	Sil.	Sep.
K-means	25.33	0.27	0.35
SC	43.93	0.41	0.58
AC	34.52	0.33	0.46
Ours	**93.42**	**0.73**	**0.87**

with the unified integration of graph augmentation, contrastive learning, and residual projection, which together balance efficiency and generalization. Moreover, the multi-metric joint optimization strategy further ensures balanced performance, enabling GCLC to remain stable and effective across domains.

C. Open-World Classification Performance

In the open-world classification task, we evaluate the model's recognition ability for unseen traffic classes. We constructed an open-world scenario based on the APPLICATIONS_21 dataset by removing three application classes from the training set (treated as new classes during testing), and compared GCLC with FlowPrint [30] and Velatin [11], the two models capable of handling open-world scenarios, in terms of new traffic identification performance. To ensure a fair comparison, all models were evaluated using the exact same raw traffic dataset. However, since our proposed graph augmentation strategy involves topological operations specific to Traffic Interaction Graphs (TIGs), it cannot be directly applied to non-graph baseline models. Therefore, FlowPrint and Velatin were evaluated using their native feature extraction pipelines and optimal default configurations as proposed in their respective original papers. This setup ensures that we are comparing the overall architectural designs and core classification logics in their intended, optimal states.

As shown in Table III, **GCLC performs excellently in the open-world setting: it achieves an accuracy of 95.02%, significantly outperforming Velatin (83.67%) and FlowPrint (57.43%), while also ranking first in precision, recall, and F1-score.**

These results indicate that GCLC possesses strong generalization and robustness in open-world classification of unseen traffic. Its advantages stem from a highly separable embedding space constructed via contrastive learning, the precise detection mechanism based on Mahalanobis distance, and a unified framework integrating new class detection and classification.

D. Clustering Performance Evaluation

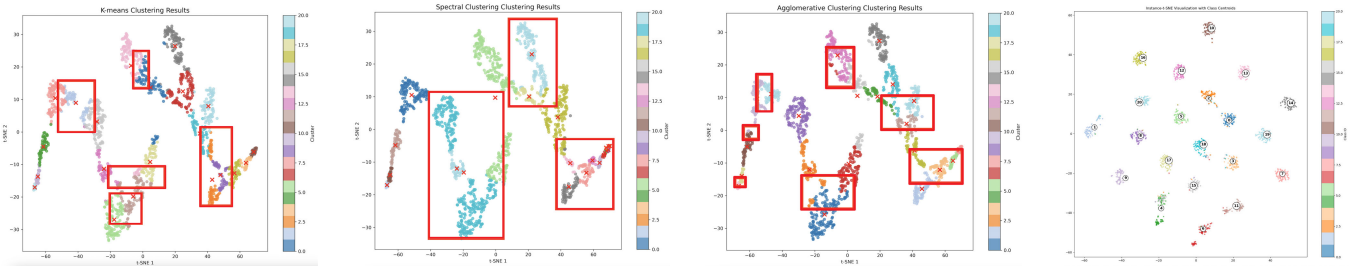
Overall Clustering Performance:

To evaluate the unsupervised clustering capability of the proposed framework, we compared it with three classical algorithms (K-means [29], Spectral Clustering (SC) [36], and Agglomerative Clustering (AC) [37]) on the APPLICATIONS_21 dataset. Using three metrics—Clustering Accuracy (ACC), Silhouette Score (Sil.), and Inter-cluster Separation (Sep.)—and verifying via clustering visualization (Fig. 7) and dynamic t-SNE visualization during training (Fig. 8), **the results show the proposed method significantly outperforms baselines across all metrics and visualization dimensions.**

The proposed method exhibits outstanding performance in all metrics (Tab. IV): it achieves a clustering accuracy of 93.42%, nearly 4 times that of K-means (25.33%) and about 2 times that of SC (43.93%); the Silhouette Score is 0.73 and Inter-cluster Separation is 0.87, both significantly higher than SC's 0.41 and 0.58. In terms of visualization, baseline algorithms show obvious cluster overlaps (marked by red boxes), while the proposed method yields compact clusters with clear boundaries (Fig. 7).

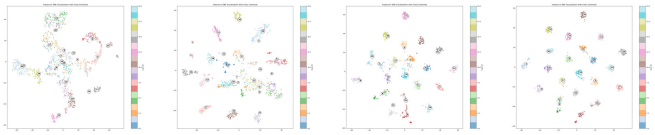
This result benefits from the integration of a supervised contrastive objective and a center-aware design, enabling tight intra-cluster cohesion and clear inter-cluster

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:11:12 UTC from IEEE Xplore. Restrictions apply.



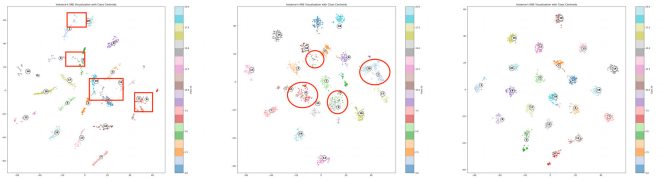
(a) K-means (b) SC (c) AC (d) Ours

Fig. 7. Clustering visualization: comparison between our model and three classical clustering algorithms.



(a) Epoch 1 (b) Epoch 10 (c) Epoch 25 (d) Epoch 40

Fig. 8. t-SNE visualization: as training proceeds, clusters become progressively compact and well-separated.



(a) SupCon-only (b) Center-only (c) SupCon + Center

Fig. 9. Ablation t-SNE on APPLICATIONS_21: combining SupCon and Center-aware losses produces the most compact and clearly separated clusters.

boundaries. Dynamic t-SNE visualization reveals that from Epoch 1 to Epoch 40, initially scattered embeddings gradually form compact and well-separated clusters (Fig. 8), intuitively verifying its excellent performance in the process from feature learning to cluster formation, as well as its advantages in intra-cluster compactness and inter-cluster separability.

Effectiveness of Supervised Contrastive and Center-aware Losses: To evaluate the contribution of the supervised contrastive loss (SupCon) and center-aware loss (Center) to clustering quality, three configurations are designed on the APPLICATIONS_21 dataset: "SupCon-only," "Center-only," and "SupCon+Center." Their performance is compared using clustering accuracy (ACC), Silhouette score (Sil.), inter-cluster separation (Sep.), and t-SNE visualization. **The results show that "SupCon+Center" performs best across all metrics, confirming that their combination can significantly improve clustering accuracy, intra-cluster compactness, and inter-cluster separability.**

As shown in Table V, "SupCon+Center" achieves an ACC of 93.42%, higher than "SupCon-only" (90.41%) and "Center-only" (89.73%), with superior Sil. and Sep. metrics. In the visualization (Fig. 9), "SupCon-only" exhibits obvious cluster overlaps (marked by red boxes), "Center-only" still has some cluster mixing (marked by red circles), while "SupCon+Center" produces compact clusters with clear boundaries and no overlaps.

TABLE V

ABLATION ON LOSS DESIGN: SUPCON-ONLY VS. CENTER-ONLY VS. BOTH (OURS) ON APPLICATIONS_21

Setting	ACC (%)	Sil.	Sep.
SupCon-only	90.41	0.67	0.61
Center-only	89.73	0.54	0.69
SupCon + Center	**93.42**	**0.73**	**0.87**

This indicates that SupCon excels at bringing similar samples closer and pushing dissimilar ones apart, but lacks sufficient inter-cluster boundaries and intra-cluster compactness. Center optimizes cluster boundaries but suffers from scattered intra-cluster samples. The combination leverages the advantages of both local alignment and global structural regularization, thereby improving clustering accuracy and separability.

E. Convergence Behavior

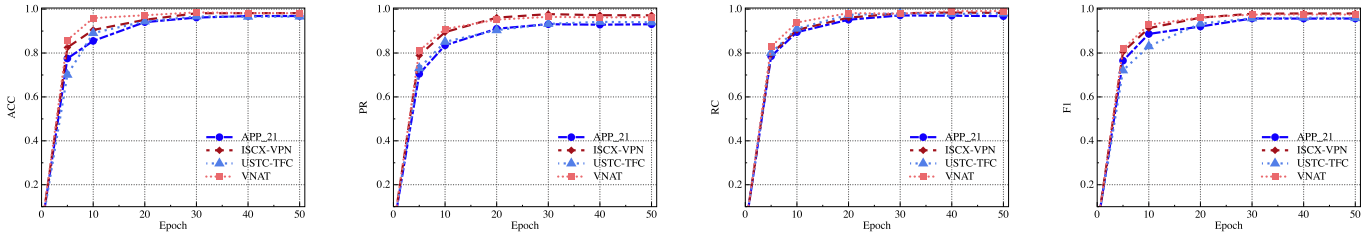
We conducted experiments on four encrypted traffic datasets, evaluating convergence using four key metrics: ACC, PR, RC, and F1. As shown in Fig. 10, bfGCLC achieves fast convergence across all datasets and metrics, demonstrating its efficient learning capability and strong generalization.

In this experiment, GCLC exhibits rapid convergence on all evaluation metrics. The model reaches over 80% accuracy by the 10th epoch and converges by the 40th epoch, demonstrating its ability to quickly learn discriminative features and achieve high class separability. Recall exceeds 90% at the 10th epoch, confirming GCLC's stable detection capability even under imbalanced data. F1 scores surpass 95% across all datasets, highlighting its robustness under diverse traffic conditions. These evaluation results validate the model's robustness, class separability, and generalization ability in various encrypted traffic scenarios.

F. Threshold Selection and Validation

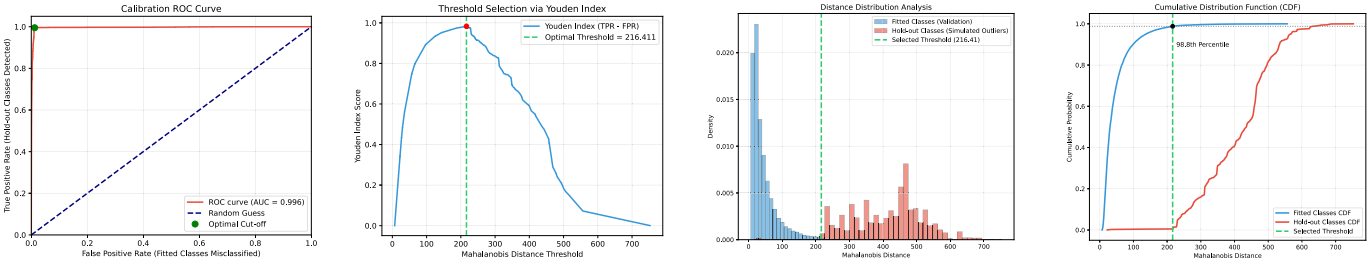
In the New Class Discoverer module (Sec. IV-E), we determine whether a sample belongs to a new traffic class by calculating its distance to the cluster centers of existing classes. Thus, an effective threshold needs to be selected to separate new classes from known classes. However, selecting this threshold using ground-truth novel classes from the test

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:11:12 UTC from IEEE Xplore. Restrictions apply.



(a) ACC (b) PR (c) RC (d) F1

Fig. 10. Performance of GCLC across four datasets and evaluation metrics.



(a) Calibration ROC Curve (b) Youden Index (c) Distance Distribution Analysis (d) Cumulative Distribution Function

Fig. 11. Threshold selection via ROC and distance distribution analysis, showing clear separation between fitted and hold-out classes.

set violates the strict open-world assumption. To address this, we implemented a zero-leakage hold-out validation strategy to determine the threshold autonomously.

First, we partitioned the known training dataset into 15 fitted classes and 3 hold-out classes (simulated as pseudo-unknowns). Using a calibration model trained solely on the fitted classes, we evaluated the Mahalanobis distances for both subsets. As shown in the calibration ROC curve (Fig. 11a and Fig. 11b), the model accurately distinguished the two sets with an AUC of 0.996. By maximizing the Youden index, we identified the optimal absolute cut-off for this calibration state at 216.411.

However, the absolute Mahalanobis distance is highly sensitive to the scale and compactness of the feature space, which naturally shifts when the final model is trained on all 18 known classes. Therefore, instead of transferring a rigid numerical value, we extract and transfer the underlying statistical decision boundary. As depicted in the cumulative distribution function (CDF) and distance distribution plots for the calibration phase (Fig. 11c and Fig. 11d), the optimal threshold of 216.411 accurately intercepts the 98.8th percentile of the fitted classes' distance distribution. This 98.8% coverage serves as our universal, scale-invariant decision strategy, balancing "known-class coverage" and "new-class misclassification risk".

For final deployment, we applied this 98.8th percentile rule to the distance distribution of the final 18-class model. In this more tightly clustered feature space, the 98.8th percentile corresponds to an adaptive global threshold of 19.76. As illustrated in the final open-set evaluation (Fig. 12), the transferred threshold of 19.76 precisely separates the density cores of the 18 known classes and the 3 completely unseen real novel

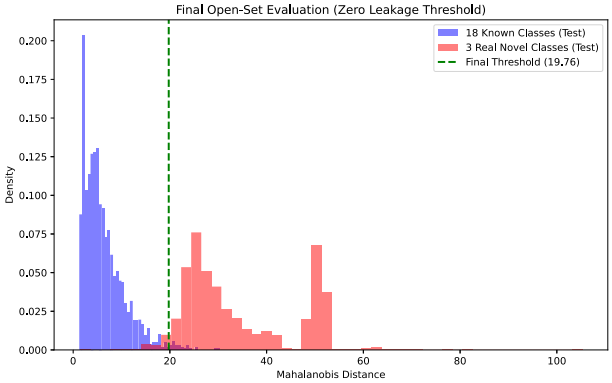


Fig. 12. Density distributions of Mahalanobis distances for known (blue) and novel (red) classes, with the zero-leakage threshold at 19.76 marked by the green dashed line.

classes. The density core of known classes lies almost entirely to the left of the threshold, while that of new classes lies to the right, ultimately achieving minimal distribution overlap. This confirms that transferring the statistical percentile strategy guarantees highly accurate class distinction while strictly adhering to zero-leakage open-world protocols.

G. Computational Complexity and Latency Analysis

While Mahalanobis detection introduces $O(d^2)$ inference and $O(d^3)$ covariance matrix update complexities, GCLC effectively mitigates these for real-time deployment. First, our GNN projects inputs into a highly compact embedding space ($d \approx 144$), rendering dense operations trivial. On an RTX 4090 GPU, average per-flow inference takes ~1.8 ms (distance calculation $<$ 0.04 ms). Second, the $O(d^3)$ matrix

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:11:12 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
ABLATION STUDY OF KEY COMPONENTS

Model	ACC	PR	Recall	F1
Full Model	95.02	94.71	96.45	95.57
w/o GA	92.61	90.01	92.91	91.44
w/o GCC	66.71	64.78	68.59	66.63
w/o NCD	88.68	78.49	91.72	84.59
w/o GA + NCD	85.38	70.72	87.11	78.06
w/o GA + GCC	49.23	44.59	51.90	47.97
w/o GCC + NCD	41.15	39.93	42.45	41.15

Abbreviations: GA: Graph Augmentation, GCC: Graph Contrastive Clustering, NCD: New Class Detector.

inversion is executed asynchronously in the background upon aggregating new-class samples. This periodic update requires merely 1.5 ms, ensuring zero blocking overhead. Thus, GCLC fully satisfies the strict low-latency requirements of dynamic networks.

H. Ablation Study

To evaluate the contribution of each core component in the framework, we conduct a systematic ablation study on the APPLICATIONS_21 dataset, removing the following modules individually or jointly:

Graph Augmentation (GA): Generates network-characteristic-aligned samples to address data scarcity, class imbalance, and robustness issues in complex networks;

Graph Contrastive Clustering (GCC): Learns class-discriminative embeddings via supervised contrastive and center-aware losses;

New Class Detector (NCD): Enables new class detection and clustering based on Mahalanobis distance to support open-world recognition.

Based on the results in Table VI, the conclusions are as follows:

GA: Removing it reduces accuracy from 95.02% to 92.61% and F1-score from 95.57% to 91.44%, verifying its role in enhancing robustness and reducing overfitting (especially in class-imbalanced scenarios);

GCC: Excluding it leads to a significant performance degradation (F1-score drops to 66.63%), confirming its core position in learning discriminative embeddings and maintaining inter-class separability;

NCD: Disabling it severely impairs the model's ability to handle open-world data (F1-score drops to 84.59%), highlighting its necessity in scenarios with frequent unknown classes.

Complementarity of Components: Removing multiple components significantly degrades performance. Removing GA and NCD together drops the F1-score to 78.06%. Furthermore, removing combinations involving GCC (e.g., GA + GCC, GCC + NCD) results in F1-scores below 50%, reflecting strong synergy. Notably, the simultaneous absence of GCC and NCD causes a near-collapse of classification performance (dropping to an F1-score of 41.15%), making both components foundational to the framework's success.

The ablation study demonstrates that each module significantly improves GCLC's performance. The integrated framework excels in accuracy, robustness, and open-world adaptability, validating the value of combining representation learning, data augmentation, and new class detection.

VI. CONCLUSION

In this paper, we propose a unified graph-based contrastive clustering framework for encrypted traffic classification in open-world environments. By modeling encrypted traffic as interaction graphs, our method enables Graph Neural Networks (GNNs) to capture the macroscopic structural and relational characteristics of network traffic. To tackle data insufficiency and class imbalance, we introduce a domain-specific graph augmentation strategy that improves data diversity and model robustness. Coupled with a supervised contrastive learning module and a center-aware loss, our framework significantly enhances feature discriminability. Furthermore, by integrating a Mahalanobis distance-based detector with an adaptive update mechanism, the framework effectively identifies both known and unseen traffic categories. Extensive evaluations on three public datasets and one self-collected dataset demonstrate that our approach outperforms state-of-the-art baselines across standard metrics, excelling particularly in clustering quality and new class discovery.

While this work provides a practical solution for dynamic and security-critical environments, we explicitly acknowledge certain theoretical boundaries in our current design. Fundamentally, lightweight packet-level features are inherently weakly discriminative in isolation; the discriminative power of our framework stems entirely from the topological representation, where the GNN aggregates local attributes into structural paradigms. Although our graph augmentation implicitly simulates and defends against real-world partial obfuscation and network jitter, it faces three specific limitations. First, absolute Constant-Bit-Rate (CBR) padding would degrade the graph into a featureless chain, neutralizing our structural approach. Second, our simplistic random node addition strategy fails to capture the intricate topological variations caused by real-world TCP fragmentation, as accurately modeling such behaviors requires stateful parameters (e.g., sequence numbers) that contradict our lightweight premise. Third, while our binary temporal discretization effectively mitigates network jitter, it inevitably sacrifices fine-grained temporal dynamics.

Nevertheless, these limitations highlight clear pathways for future open-world network intelligence research. Future efforts will address these boundaries by exploring learnable, time-aware attention mechanisms and more sophisticated, protocol-aware graph augmentation strategies, ultimately enhancing model robustness in highly fragmented and ultra-obfuscated environments.

REFERENCES

[1] R. Xie et al., "ModelFreeUP: Attacking sparse network monitoring via model-free universal adversarial perturbation," *IEEE Trans. Netw.*, no. 99, Jan. 2026.

[2] J. Tian et al., "Matrix reshaping for reduced sensing cost and improved data inference in sparse mobile sensing environments," *IEEE Trans. Mobile Comput.*, vol. 25, no. 4, pp. 5253–5266, Apr. 2026.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:11:12 UTC from IEEE Xplore. Restrictions apply.

[3] Internet Assigned Numbers Authority (IANA). *Service Name and Transport Protocol Port Number Registry*. Accessed: Jul. 1, 2020. [Online]. Available: https://www.iana.org/assignments/service-names-port-numbers/service-names-port-numbers.xhtml

[4] A. Azab, M. Khasawneh, S. Alrabaee, K.-K.-R. Choo, and M. Sarsour, "Network traffic classification: Techniques, datasets, and challenges," *Digit. Commun. Netw.*, vol. 10, no. 3, pp. 676–692, Jun. 2024.

[5] J. Zhang, F. Li, F. Ye, and H. Wu, "Autonomous unknown-application filtering and labeling for DL-based traffic classifier update," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, Jul. 2020, pp. 397–405.

[6] N. Williams, S. Zander, and G. Armitage, "*Evaluating machine learning algorithms for automated network application identification*," Centre Adv. Internet Archit. (CAIA), Swinburne Univ. Technol., Melbourne, VIC, Australia, Tech. Rep. 060410B, Mar. 2006.

[7] A. Este, F. Gringoli, and L. Salgarelli, "Support vector machines for TCP traffic classification," *Comput. Netw.*, vol. 53, no. 14, pp. 2476–2490, Sep. 2009.

[8] R. Alshammari and A. N. Zincir-Heywood, "An investigation on the identification of VoIP traffic: Case study on gtalk and skype," in *Proc. Int. Conf. Netw. Service Manage.*, Oct. 2010, pp. 310–313.

[9] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[10] R. Zhao et al., "Yet another traffic classifier: A masked autoencoder based traffic transformer with multi-level flow representation," in *Proc. AAAI Conf. Artif. Intell.*, 2023, vol. 37, no. 4, pp. 5420–5427.

[11] S. Le, Y. Lai, Y. Wang, and H. He, "An adaptive classification and updating method for unknown network traffic in open environments," *Comput. Netw.*, vol. 238, Jan. 2024, Art. no. 110114.

[12] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "AppScanner: Automatic fingerprinting of smartphone apps from encrypted network traffic," in *Proc. IEEE Eur. Symp. Secur. Privacy (EuroSP)*, Mar. 2016, pp. 439–454.

[13] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-net: A flow sequence network for encrypted traffic classification," in *Proc. IEEE Conf. Comput. Commun.*, Apr. 2019, pp. 1171–1179.

[14] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Accurate decentralized application identification via encrypted traffic analysis using graph neural networks," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2367–2380, 2021.

[15] M. Jiang et al., "Accurate mobile-app fingerprinting using flow-level relationship with graph neural networks," *Comput. Netw.*, vol. 217, Nov. 2022, Art. no. 109309.

[16] J. Yang, X. Jiang, Y. Lei, W. Liang, Z. Ma, and S. Li, "MTSecurity: Privacy-preserving malicious traffic classification using graph neural network and transformer," *IEEE Trans. Netw. Service Manage.*, vol. 21, no. 3, pp. 3583–3597, Jun. 2024.

[17] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.

[18] P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio, and P.-A. Manzagol, "Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion," *J. Mach. Learn. Res.*, vol. 11, no. 110, pp. 3371–3408, 2010.

[19] F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain," *Psychol. Rev.*, vol. 65, no. 6, pp. 386–408, 1958.

[20] J. Zhou, W. Fu, W. Hu, Z. Sun, T. He, and Z. Zhang, "Challenges and advances in analyzing TLS 1.3-encrypted traffic: A comprehensive survey," *Electronics*, vol. 13, no. 20, p. 4000, Oct. 2024.

[21] J.-P. Smith, L. Dolfi, P. Mittal, and A. Perrig, "QCSD: A QUICclient-sidewebsite-fingerprinting defence framework," in *Proc. 31st USENIX Secur. Symp. (USENIX Security)*, 2022, pp. 771–789.

[22] Z. Zhao, Y. Guo, J. H. Wang, H. Wang, C. Zhang, and C. An, "CL-ETC: A contrastive learning method for encrypted traffic classification," in *Proc. IFIP Netw. Conf. (IFIP Networking)*, Jun. 2022, pp. 1–9.

[23] H. Zhang, X. Xiao, L. Yu, Q. Li, Z. Ling, and Y. Zhang, "One train for two tasks: An encrypted traffic classification framework using supervised contrastive learning," 2024, *arXiv:2402.07501*.

[24] S. Huang, K. Chen, C. Liu, A. Liang, and H. Guan, "A statistical-feature-based approach to internet traffic classification using machine learning," in *Proc. Int. Conf. Ultra Modern Telecommun. Workshops*, Oct. 2009, pp. 1–6.

[25] C. Cortes and V. Vapnik, "Support-vector networks," *Mach. Learn.*, vol. 20, no. 3, pp. 273–297, 1995.

[26] D. D. Lewis, "Naive (Bayes) at forty: The independence assumption in information retrieval," in *Proc. Eur. Conf. Mach. Learn.* Berlin, Germany: Springer, 1998, pp. 4–15.

[27] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[28] S. L. Salzberg and J. R. Quinlan, *C4. 5: Programs for Machine Learning*. San Mateo, CA, USA: Morgan Kaufmann, 1994.

[29] A. Likas, N. Vlassis, and J. J. Verbeek, "The global k-means clustering algorithm," *Pattern Recognit.*, vol. 36, no. 2, pp. 451–461, Feb. 2003.

[30] T. van Ede et al., "FlowPrint: Semi-supervised mobile-app fingerprinting on encrypted network traffic," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2020, pp. 1–18.

[31] P. C. Mahalanobis, "On the generalised distance in statistics," *Proc. Nat. Inst. Sci. India*, vol. 2, no. 1, pp. 49–55, 1936.

[32] Y. Yin et al., "GraphIoT: Lightweight IoT device detection based on graph classifiers and incremental learning," *IEEE Trans. Services Comput.*, vol. 17, no. 6, pp. 3758–3772, Nov. 2024.

[33] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related features," in *Proc. 2nd Int. Conf. Inf. Syst. Secur. Privacy*, Portugal, 2016, pp. 407–414.

[34] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Da Nang, Vietnam, Jan. 2017, pp. 712–717.

[35] S. Jorgensen et al., "Extensible machine learning for encrypted network traffic application labeling via uncertainty quantification," *IEEE Trans. Artif. Intell.*, vol. 5, no. 1, pp. 420–433, Jan. 2024.

[36] J. Shi and J. Malik, "Normalized cuts and image segmentation," *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, vol. 22, no. 8, pp. 888–905, 2000.

[37] K. Chidananda Gowda and G. Krishna, "Agglomerative clustering using the concept of mutual nearest neighbourhood," *Pattern Recognit.*, vol. 10, no. 2, pp. 105–112, Jan. 1978.



Jigang Wen received the Ph.D. degree in computer application from Hunan University, Changsha, China, in 2011. He was a Research Assistant with the Department of Computing, The Hong Kong Polytechnic University, Hong Kong, from 2008 to 2010. He is currently with the School of Computer Science and Engineering, Hunan University of Science and Technology, Xiangtan, China. His research interests include wireless network and mobile computing and high-speed network measurement and management.



Xiyu Zhang received the B.E. degree from Shaoguan University in 2023. She is currently pursuing the M.S. degree with the School of Computer Science and Engineering, Hunan University of Science and Technology. Her research interests include traffic identification, contrastive learning, network security, and graph neural networks.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:11:12 UTC from IEEE Xplore. Restrictions apply.



Kun Xie (Member, IEEE) received the Ph.D. degree in computer applications from Hunan University in 2007 and subsequently conducted Post-Doctoral Research with The Hong Kong Polytechnic University. She is currently a second-level Professor and a Doctoral Supervisor with Hunan University, a Director of the Key Laboratory of Supercomputing and Artificial Intelligence Converged Computing, Ministry of Education, and a Chair of both the Academic Committee and the Degree Committee of the College of Computer Science and Electronic Engineering with Hunan University. She has published more than 170 papers in top-tier conferences and journals, including SIGMOD, CCS, INFOCOM, ICDE, SIGMETRICS, IEEE/ACM TRANSACTIONS ON NETWORKING, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, IEEE TRANSACTIONS ON SERVICES COMPUTING, IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING, and IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING. Her long-standing research interests include matrix/tensor completion, computer networks, network security, and artificial intelligence.



Yuxiang Zeng received the bachelor's degree from Shantou University in 2024. He is currently pursuing the master's degree with Hunan University, China, under the supervision of Prof. Kun Xie. During his undergraduate studies, he has published five papers, including those in CCF C-ranked conferences, such as IEEE SCC/SSE and IEEE ICPADS. His research interests include deep learning, data mining, and web services.



Kun Huang received the B.E., M.E., and Ph.D. degrees in computer science and technology from Hunan University, in 2001, 2004, and 2009, respectively. He is currently an Associate Professor with Peng Cheng Laboratory, China. His research interests include programmable networks, datacenter networks, and emerging networks.



Yudian Ouyang received the Ph.D. degree in computer science and technology from Hunan University, Changsha, China, in 2024. She is currently a Lecturer with Hunan Normal University. Her research interests include network measurement and analysis, tensor completion, and deep learning.



Wei Liang received the Ph.D. degree in computer science and technology from Hunan University, Changsha, China, in 2013. He was a Post-Doctoral Scholar with Lehigh University, Bethlehem, PA, USA, from 2014 to 2016. He is currently a Professor and the Dean of the School of Computer Science and Engineering, Hunan University of Science and Technology, Xiangtan, China. His research interests include blockchain security technology, network security protection, embedded systems, and hardware IP protection.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:11:12 UTC from IEEE Xplore. Restrictions apply.